

คู่มือสร้างเว็บแบบเร่งด่วน

แนวคิดหลัก: ไม่ว่าโจทย์วันสอบจริงจะสั่งให้สร้างระบบอะไร (ระบบนักเรียน, ระบบสินค้า, ระบบยืม-คืนหนังสือ, ระบบลงทะเบียนกิจกรรม ฯลฯ) **โครงสร้างเบื้องหลังจะเหมือนกันหมด** คือรูปแบบ **CRUD** (Create, Read, Update, Delete) บนตารางเดียว ต่างกันแค่ "ชื่อตาราง" และ "ชื่อฟิลด์" เท่านั้น ตามที่ขอบเขตสอบระบุไว้ (PHP + MySQL + HTML/CSS/Bootstrap + JavaScript/jQuery + AJAX) เอกสารนี้จึงให้ **แม่แบบโค้ดที่ทำงานได้จริง** ใช้ตัวอย่างระบบ "นักเรียน" เป็นหลัก — เมื่อรู้โจทย์จริงแล้ว แค่แก้ชื่อตาราง/ฟิลด์ให้ตรงกันทุกไฟล์ก็นำไปใช้ได้ทันที

ข้อควรระวัง: โปรดสอบถามกรรมการคุมสอบล่วงหน้าว่าอนุญาตให้นำเอกสารอ้างอิง/ไฟล์แม่แบบเข้าห้องสอบได้หรือไม่ และห้องสอบมีอินเทอร์เน็ตหรือไม่ (มีผลต่อการใช้ Bootstrap/jQuery แบบ CDN) เอกสารนี้จัดทำเพื่อให้ฝึกพิมพ์ซ้ำจนจำได้ **ขึ้นใจ** เป็นหลัก ไม่ใช่สิ่งที่รับประกันว่านำเข้าห้องสอบได้

สารบัญ

- 1. แผนบริหารเวลา 3 ชั่วโมง
- 2. โครงสร้างไฟล์ที่แนะนำ + เตรียมฐานข้อมูล
- 3. Step 1: ไฟล์เชื่อมต่อฐานข้อมูล (connect.php)
- 4. Step 2: หน้าหลัก — ฟอรัมเพิ่มข้อมูล + แสดงตาราง (index.php)
- 5. Step 3: ใส่ Bootstrap ให้ Responsive
- 6. Step 4: ค้นหาแบบ Real-time ด้วย AJAX
- 7. Step 5: ลบข้อมูลด้วย AJAX + Confirm
- 8. Step 6: แก้ไขข้อมูลด้วย Modal + AJAX
- 9. เช็คลิสต์ก่อนส่งงาน
- 10. ตารางปรับใช้กับโจทย์อื่น (ถ้าไม่ใช่ระบบนักเรียน)

1. แผนบริหารเวลา 3 ชั่วโมง (180 นาที)

ช่วงเวลา	สิ่งที่ต้องทำ
0 - 10 นาที	อ่านโจทย์ให้ครบ วางแผนว่าตารางหลักชื่ออะไร มีฟิลด์อะไรบ้าง (เขียนใส่กระดาษก่อนลงมือพิมพ์)
10 - 25 นาที	สร้างฐานข้อมูลและตารางผ่าน phpMyAdmin หรือรันคำสั่ง SQL ตาม Step 1
25 - 60 นาที	เขียน PHP หลัก: เชื่อมต่อฐานข้อมูล + insert ข้อมูลจากฟอรัม + select แสดงรายการ (ยังไม่ต้องสวย เน้นให้ทำงานถูกต้องก่อน)
60 - 90 นาที	ใส่ Bootstrap ครอบฟอรัมและตารางให้ดูดีและ responsive
90 - 120 นาที	ทำระบบค้นหาแบบ Real-time ด้วย AJAX (jQuery keyup)
120 - 150 นาที	ทำปุ่มลบด้วย AJAX พร้อมกล่องยืนยันก่อนลบ

150 - 175 นาที	ทำปุ่มแก้ไขแบบ Modal + AJAX
175 - 180 นาที	ทดสอบทุกปุ่มอีกครั้ง ตรวจสอบ error เตรียมส่งงาน

หลักคิด: ทำให้ระบบ "ทำงานได้" (functional) ให้เสร็จก่อนเสมอ แล้วค่อยเพิ่มความสวยงามและ AJAX ทีหลัง เพราะกรรมการให้คะแนนความถูกต้องของฟังก์ชันเป็นหลัก ถ้าเวลาไม่พอ อย่างน้อยควรมี CRUD พื้นฐานที่ใช้งานได้ครบ (แม้ยังไม่มี AJAX ก็ยังได้คะแนนในส่วน PHP+MySQL+HTML)

2. โครงสร้างไฟล์ที่แนะนำ + เตรียมฐานข้อมูล

แนะนำให้แยกไฟล์ดังนี้ (ใช้ตัวอย่างระบบนักเรียน):

```
project/
├─ connect.php      (เชื่อมต่อฐานข้อมูล ใช้ include ในทุกไฟล์)
├─ index.php        (หน้าแรก: ฟอรั่มเพิ่ม + ตารางแสดงผล + ช่องค้นหา + modal แก้ไข)
├─ search.php       (AJAX endpoint สำหรับค้นหา)
├─ delete.php       (AJAX endpoint สำหรับลบ)
├─ get_one.php      (AJAX endpoint ดึงข้อมูล 1 แถวมาแสดงใน modal แก้ไข)
└─ update.php       (AJAX endpoint สำหรับบันทึกการแก้ไข)
```

สร้างฐานข้อมูลและตาราง (รันใน phpMyAdmin แท็บ SQL)

```
CREATE DATABASE IF NOT EXISTS school_db;
USE school_db;

CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT,
    class VARCHAR(20)
);
```

ตั้งชื่อฐานข้อมูลและตารางให้ตรงกับโจทย์จริงที่ได้รับในวันสอบ เช่น ถ้าโจทย์ให้ทำระบบสินค้า อาจเปลี่ยนเป็น `shop_db` และตาราง `products (id, name, price, quantity)`

3. Step 1: ไฟล์เชื่อมต่อฐานข้อมูล

connect.php

```
<?php
$conn = mysqli_connect("localhost", "root", "", "school_db");
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
?>
```

ไฟล์นี้จะถูก `include` หรือ `require` เข้าไปในทุกไฟล์ PHP ที่ต้องคุยกับฐานข้อมูล เขียนครั้งเดียวใช้ได้ทั้งโปรเจกต์ ถ้าเปลี่ยนชื่อฐานข้อมูล ให้แก้ที่ไฟล์นี้ไฟล์เดียว

4. Step 2: หน้าหลัก — ฟอรั่มเพิ่มข้อมูล + แสดงตาราง

index.php

```
<?php
require "connect.php";

// รับข้อมูลจากฟอร์มและบันทึก (ทำงานเมื่อกด submit เท่านั้น)
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name'];
    $age = $_POST['age'];
    $class = $_POST['class'];

    $stmt = $conn->prepare("INSERT INTO students (name, age, class) VALUES (?, ?, ?)");
    $stmt->bind_param("sis", $name, $age, $class);
    $stmt->execute();
}

// ดึงข้อมูลทั้งหมดมาแสดง
$result = $conn->query("SELECT * FROM students ORDER BY id DESC");
?>

<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="UTF-8">
    <title>ระบบจัดการนักเรียน</title>
</head>
<body>
    <h2>เพิ่มข้อมูลนักเรียน</h2>
    <form method="post">
        <input type="text" name="name" placeholder="ชื่อ" required>
        <input type="number" name="age" placeholder="อายุ" required>
        <input type="text" name="class" placeholder="ชั้นเรียน" required>
        <button type="submit">บันทึก</button>
    </form>

    <h2>รายชื่อนักเรียน</h2>
    <table border="1">
        <tr><th>ชื่อ</th><th>อายุ</th><th>ชั้น</th></tr>
        <?php while ($row = $result->fetch_assoc()) { ?>
        <tr>
            <td><?= htmlspecialchars($row['name']) ?></td>
            <td><?= htmlspecialchars($row['age']) ?></td>
            <td><?= htmlspecialchars($row['class']) ?></td>
        </tr>
        <?php } ?>
    </table>
</body>
</html>
```

นี่คือระบบ CRUD พื้นฐานที่ "ทำงานได้จริง" แล้ว — ยังไม่มี Bootstrap หรือ AJAX แต่ครบ PHP+MySQL+HTML ตามขอบเขตสอบ ถ้าเวลาเหลือน้อยมาก ระบบระดับนี้ยังพอทำคะแนนได้ ควรทำให้ถึงจุดนี้ให้เร็วที่สุดก่อนเสมอ

5. Step 3: ใส่ Bootstrap ให้ Responsive

เพิ่ม CDN ของ Bootstrap และ jQuery ใน `<head>` (หากห้องสอบมีอินเทอร์เน็ต) หรือดาวน์โหลดไฟล์ .css/.js มาเก็บไว้ในเครื่องล่วงหน้าแล้วอ้างอิงแบบ local:

```
<head>
  <meta charset="UTF-8">
  <title>ระบบจัดการนักเรียน</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
</head>
```

ครอบฟอร์มและตารางด้วย class ของ Bootstrap:


```

<div class="container mt-4">
  <h2>เพิ่มข้อมูลนักเรียน</h2>
  <form method="post" class="row g-2">
    <div class="col-md-4">
      <input type="text" name="name" class="form-control" placeholder="ชื่อ" required>
    </div>
    <div class="col-md-3">
      <input type="number" name="age" class="form-control" placeholder="อายุ" required>
    </div>
    <div class="col-md-3">
      <input type="text" name="class" class="form-control" placeholder="ชั้นเรียน" required>
    </div>
    <div class="col-md-2">
      <button type="submit" class="btn btn-primary w-100">บันทึก</button>
    </div>
  </form>

  <h2 class="mt-4">รายชื่อนักเรียน</h2>
  <input type="text" id="searchBox" class="form-control mb-2" placeholder="ค้นหาชื่อนักเรียน...">
  <table class="table table-striped table-bordered">
    <thead class="table-dark">
      <tr><th>ชื่อ</th><th>อายุ</th><th>ชั้น</th><th>จัดการ</th></tr>
    </thead>
    <tbody id="studentTable">
      <?php while ($row = $result->fetch_assoc()) { ?>
        <tr id="row-<?=$row['id'] ?>">
          <td class="col-name"><?=$row['name'] ?></td>
          <td><?=$row['age'] ?></td>
          <td><?=$row['class'] ?></td>
          <td>
            <button class="btn btn-sm btn-warning btn-edit" data-id="<?=$row['id'] ?>">แก้ไข</button>
            <button class="btn btn-sm btn-danger btn-delete" data-id="<?=$row['id'] ?>">ลบ</button>
          </td>
        </tr>
      <?php } ?>
    </tbody>
  </table>
</div>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>

```

`class="row g-2"` กับ `col-md-*` คือระบบ grid ของ Bootstrap ที่ทำให้ฟอร์มเรียงตัวเป็นแนวนอนบนจอใหญ่ และเรียงตัวเป็นแนวตั้งอัตโนมัติบนจอเล็ก (responsive) — สังเกตว่าใส่ `id="row-<?=$row['id'] ?>"` ไว้ที่แต่ละแถวและใส่ `data-id` ไว้ที่ปุ่ม เพื่อให้ jQuery ใน Step ถัดไปอ้างอิงแถวนั้นๆ ได้

6. Step 4: ค้นหาแบบ Real-time ด้วย AJAX

search.php

```
<?php
require "connect.php";
$keyword = $_POST['keyword'];

$stmt = $conn->prepare("SELECT * FROM students WHERE name LIKE ?");
$search = "%$keyword%";
$stmt->bind_param("s", $search);
$stmt->execute();
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
    echo "<tr id='row-{$row['id']}'>
        <td class='col-name'>" . htmlspecialchars($row['name']) . "</td>
        <td>" . htmlspecialchars($row['age']) . "</td>
        <td>" . htmlspecialchars($row['class']) . "</td>
        <td>
            <button class='btn btn-sm btn-warning btn-edit' data-
id='{$row['id']}'>แก้ไข</button>
            <button class='btn btn-sm btn-danger btn-delete' data-id='{$row['id']}'>ลบ</button>
        </td>
    </tr>";
}
?>
```

script.js (วางท้าย index.php ก่อน </body>)

```
<script>
$(document).ready(function(){
    $('#searchBox').on('keyup', function(){
        $.ajax({
            url: 'search.php',
            method: 'POST',
            data: { keyword: $(this).val() },
            success: function(response){
                $('#studentTable').html(response);
            }
        });
    });
});
</script>
```

ทุกครั้งที่พิมพ์ในช่องค้นหา (event `keyup`) jQuery จะยิง AJAX ไปที่ `search.php` พร้อมคำค้น แล้วนำ HTML ที่ได้กลับมาแทนที่เนื้อหาใน `<tbody id="studentTable">` แทนที่โดยไม่ reload หน้า — ต้องใช้ `id` เดียวกันระหว่าง PHP ที่ generate แลวกับ jQuery ที่เขียนแทนที่เนื้อหา

7. Step 5: ลบข้อมูลด้วย AJAX + Confirm

delete.php

```
<?php
require "connect.php";
$id = $_POST['id'];

$stmt = $conn->prepare("DELETE FROM students WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();

echo "success";
?>
```

script.js (เพิ่มต่อจากเดิม)

```
<script>
$(document).on('click', '.btn-delete', function(){
    if (confirm('ยืนยันการลบข้อมูลนี้หรือไม่?')) {
        let id = $(this).data('id');
        let btn = $(this);
        $.post('delete.php', { id: id }, function(response){
            if (response.trim() === 'success') {
                btn.closest('tr').remove();
            }
        });
    }
});
</script>
```

ใช้ `$(document).on('click', '.btn-delete', ...)` แทน `$('.btn-delete').click(...)` เพราะแถวที่ได้จาก AJAX ค้นหา (Step 4) เป็นปุ่มที่ถูกสร้างขึ้นใหม่หลังหน้าโฮลดเสร็จแล้ว — ถ้าใช้ `.click()` ตรงๆ ปุ่มที่มาจากผลค้นหาจะไม่ถูกผูก event ไว้ ต้องใช้ event delegation ผ่าน document แทนเสมอ

8. Step 6: แก้ไขข้อมูลด้วย Modal + AJAX

เพิ่ม Bootstrap Modal ไว้ใน index.php (นอก loop ตาราง):

```
<div class="modal fade" id="editModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">แก้ไขข้อมูลนักเรียน</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <input type="hidden" id="edit_id">
        <input type="text" id="edit_name" class="form-control mb-2" placeholder="ชื่อ">
        <input type="number" id="edit_age" class="form-control mb-2" placeholder="อายุ">
        <input type="text" id="edit_class" class="form-control mb-2" placeholder="ชั้นเรียน">
      </div>
      <div class="modal-footer">
        <button class="btn btn-primary" id="btnSaveEdit">บันทึก</button>
      </div>
    </div>
  </div>
</div>
```

get_one.php

```
<?php
require "connect.php";
$id = $_POST['id'];

$stmt = $conn->prepare("SELECT * FROM students WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$row = $stmt->get_result()->fetch_assoc();

echo json_encode($row);
?>
```

update.php

```

<?php
require "connect.php";
$id    = $_POST['id'];
$name  = $_POST['name'];
$age   = $_POST['age'];
$class = $_POST['class'];

$stmt = $conn->prepare("UPDATE students SET name=?, age=?, class=? WHERE id=?");
$stmt->bind_param("sisi", $name, $age, $class, $id);
$stmt->execute();

echo "success";
?>

```

script.js (เพิ่มต่อจากเดิม)

```

<script>
// เปิด modal พร้อมดึงข้อมูลเดิมมาแสดง
$(document).on('click', '.btn-edit', function(){
    let id = $(this).data('id');
    $.post('get_one.php', { id: id }, function(data){
        let student = JSON.parse(data);
        $('#edit_id').val(student.id);
        $('#edit_name').val(student.name);
        $('#edit_age').val(student.age);
        $('#edit_class').val(student.class);
        new bootstrap.Modal(document.getElementById('editModal')).show();
    });
});

// บันทึกการแก้ไข
$('#btnSaveEdit').click(function(){
    $.post('update.php', {
        id: $('#edit_id').val(),
        name: $('#edit_name').val(),
        age: $('#edit_age').val(),
        class: $('#edit_class').val()
    }, function(response){
        if (response.trim() === 'success') {
            let id = $('#edit_id').val();
            $('#row-' + id + ' .col-name').text($('#edit_name').val());
            bootstrap.Modal.getInstance(document.getElementById('editModal')).hide();
        }
    });
});
</script>

```

ขั้นตอนคือ: กดปุ่มแก้ไข → AJAX ไปดึงข้อมูลปัจจุบันจาก get_one.php (คืนกลับมาเป็น JSON) → เติมค่าใน input ของ modal ด้วย `JSON.parse()` → เปิด modal ด้วย `new bootstrap.Modal(...).show()` → ผู้ใช้แก้ไขแล้วกดบันทึก → AJAX ไปที่ `update.php` → เมื่อสำเร็จอัปเดตข้อมูลในตารางทันทีด้วย `.text()` แล้วปิด modal ด้วย `bootstrap.Modal.getInstance(...).hide()` ทั้งหมดนี้ไม่มีการ reload หน้าเลย

9. เช็คลิสต์ก่อนส่งงาน

รายการตรวจสอบ	ทำแล้ว?
เพิ่มข้อมูลผ่านฟอร์มได้ และข้อมูลไปโพลในฐานข้อมูลจริง	☐
ตารางแสดงข้อมูลล่าสุดถูกต้องครบทุกแถว	☐
หน้าเว็บแสดงผลดีทั้งจอคอมพิวเตอร์และจอมือถือ (ลองย่อขนาดหน้าต่างดู)	☐
ค้นหาแล้วผลลัพธ์อัปเดตโดยไม่ reload หน้า	☐
ลบข้อมูลมีกล่องยืนยันก่อนลบจริง และแถวหายไปทันทีหลังลบสำเร็จ	☐
แก้ไขข้อมูลแล้วค่าที่แสดงในตารางเปลี่ยนตามจริง	☐
ทุก query ที่รับค่าจากผู้ใช้ใช้ prepared statement (ป้องกัน SQL Injection)	☐
ทุกจุดที่ echo ค่าจากฐานข้อมูลออกหน้าเว็บ ผ่าน htmlspecialchars() แล้ว	☐
ไม่มี error สีแดงของ PHP ปรากฏบนหน้าเว็บ	☐

10. ตารางปรับใช้กับโจทย์อื่น (ถ้าไม่ใช่ระบบนักเรียน)

แม่แบบโค้ดทั้งหมดข้างต้นใช้ได้กับทุกหัวข้อ เพียงเปลี่ยน 3 อย่างนี้ให้ตรงกันในทุกไฟล์: ชื่อฐานข้อมูล, ชื่อตาราง, ชื่อฟิลด์

หัวข้อที่อาจออกโจทย์	ชื่อตารางแนะนำ	ฟิลด์ตัวอย่าง
ระบบนักเรียน (ตัวอย่างในเอกสารนี้)	students	id, name, age, class
ระบบสินค้า/คลังสินค้า	products	id, name, price, quantity
ระบบยืม-คืนหนังสือ	books	id, title, author, status
ระบบลงทะเบียนกิจกรรม	registrations	id, fullname, event_name, phone
ระบบข้อมูลพนักงาน	employees	id, name, position, salary

เวลาปรับใช้จริง ให้ทำตามลำดับ: (1) แก่คำสั่ง CREATE TABLE ให้ตรงชื่อฟิลด์ใหม่ (2) แก่ชื่อ input ในฟอร์มให้ตรงกับ `$_POST` ที่ใช้ (3) แก่คำสั่ง INSERT/SELECT/UPDATE/DELETE ให้ใช้ชื่อฟิลด์ใหม่ (4) แก่ label ข้อความในหน้าเว็บให้ตรงหัวข้อ — โครงสร้างไฟล์และรูปแบบ AJAX ทั้งหมดไม่ต้องเปลี่ยนเลย

จัดทำเพื่อการฝึกฝนส่วนตัว — แนะนำให้ฝึกพิมพ์แม่แบบนี้ซ้ำๆ ด้วยมือเปล่าจนคล่อง (ไม่เปิดดู) เพื่อจำลองสถานการณ์จริงในห้องสอบ